



Module 2 : FST - Morphological Analysis



Agenda

- Revision : FSA
- Finite State Transition (FST)
- Examples

Finite State Automata

- Regular Expression is one-way of describing a FSA
- Any finite state automaton can be described with a RE
- A RE is one way of characterizing a particular kind of formal language called a Regular Language
- Both RE and FSA can be used to describe Regular Languages.
- A non-deterministic finite automaton is a 5-tuple (Q, S, d, q_0, F) , where
 - Q : a finite set called the states
 - Σ : a finite set called the alphabet
 - $\delta : Q \times \Sigma \rightarrow Q$ is the transition function
 - $q_0 \in Q$ is the start state
 - $F \subseteq Q$ is the set of accept states

Finite State Automata: Definition

■ $M_1 = (Q, \Sigma, \delta, q_0, F)$

■ $Q = \{q_1, q_2, q_3\}$

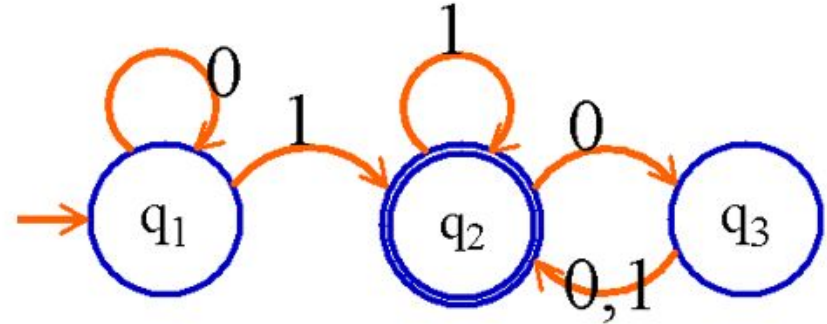
■ $\Sigma = \{0, 1\}$

■ $\delta :$

	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

■ q_1 : the start state

■ $F = \{q_2\}$



Finite State Automata: Formal Definition

- $M = (Q, \Sigma, \delta, q_0, F)$

w : a string over Σ , $w_i \in \Sigma$, $w = w_1 w_2 \dots w_n$

- M accepts w if a sequence of states $r_0, r_1, \dots, r_n$ exists in Q and satisfies the following 3 conditions:

- $r_0 = q_0$

- $\delta(r_i, w_{i+1}) = r_{i+1}$, for $i = 0, \dots, n-1$

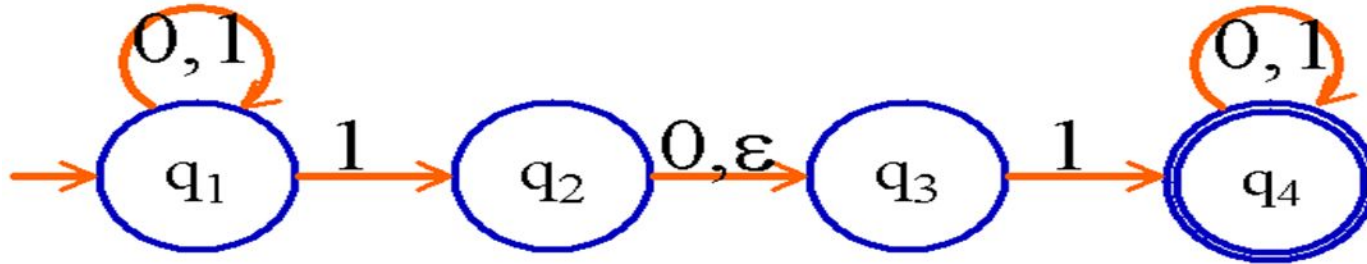
- $r_n \in F$

Finite State Automata: NFA Example

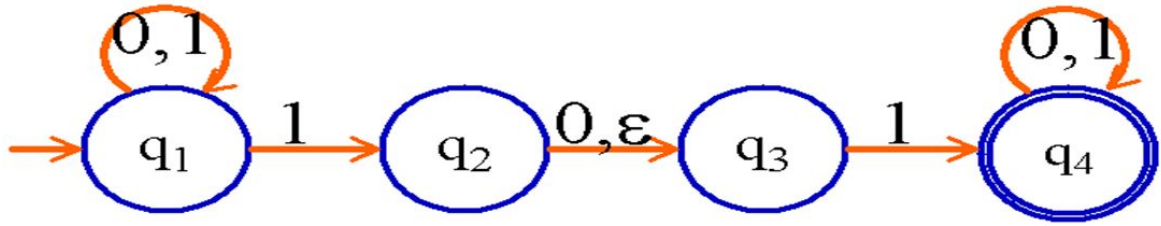
- We say that M recognizes language A if $A = \{w \mid M \text{ accepts } w\}$

Definition:

- A language is called a **regular language** if some finite automata recognizes it.



NFA Example



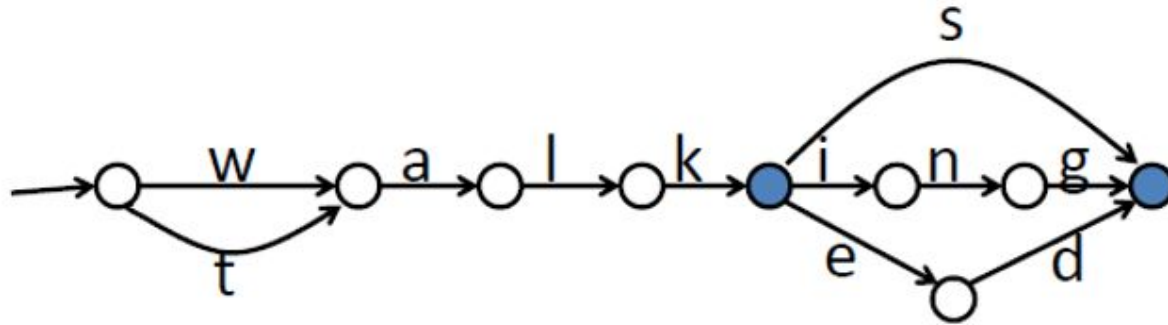
■ eg.

- $Q = \{q_1, q_2, q_3, q_4\}$
- $\Sigma = \{0, 1\}$
- $\delta :$

	0	1	ϵ
q_1	$\{q_1\}$	$\{q_1, q_2\}$	ϕ
q_2	$\{q_3\}$	ϕ	$\{q_3\}$
q_3	ϕ	$\{q_4\}$	ϕ
q_4	$\{q_4\}$	$\{q_4\}$	ϕ
- q_1 : start state
- q_4 : accept state

Finite State Automaton

- Directed graph with labelled transitions, a start state and a set of final states



- Recognises **walk, walks, walked, walking, talk, talks, talked, talking**

Finite State Automaton

- FSAs are isomorphic to regular expressions and regular grammars.

All of them define a regular language.

- **Regular expression:** $(w|t)alk(s|ed|ing)?$

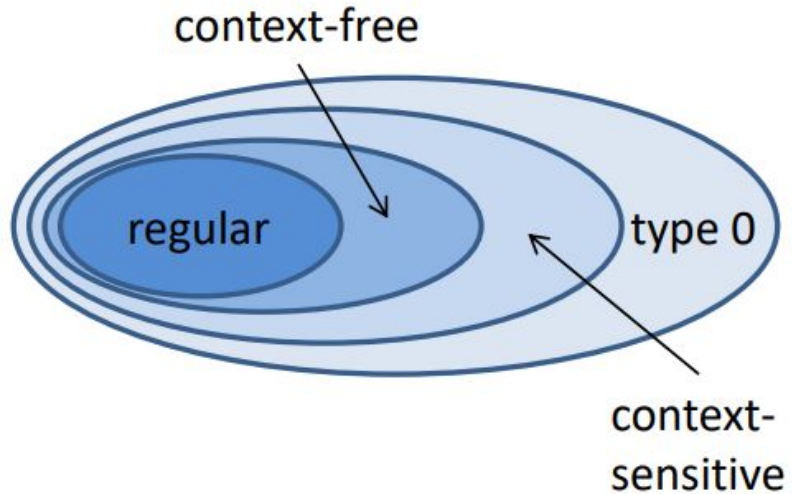
- **Regular grammar:**

- $S \rightarrow w A$
- $B \rightarrow s$ $B \rightarrow$
- $S \rightarrow t A$ $B \rightarrow e d$
- $A \rightarrow a l k B$ $B \rightarrow i n g$

Both equivalent to the automaton on the previous slide

Finite State Automaton (FSAs)

- **Regular expression:** $(w|t)alk(s|ed|ing)?$
- **Regular grammar:**
 - $S \rightarrow w A$
 - $B \rightarrow s \quad B \rightarrow$
 - $S \rightarrow t A \quad B \rightarrow e d$
 - $A \rightarrow a l k B \quad B \rightarrow i n g$

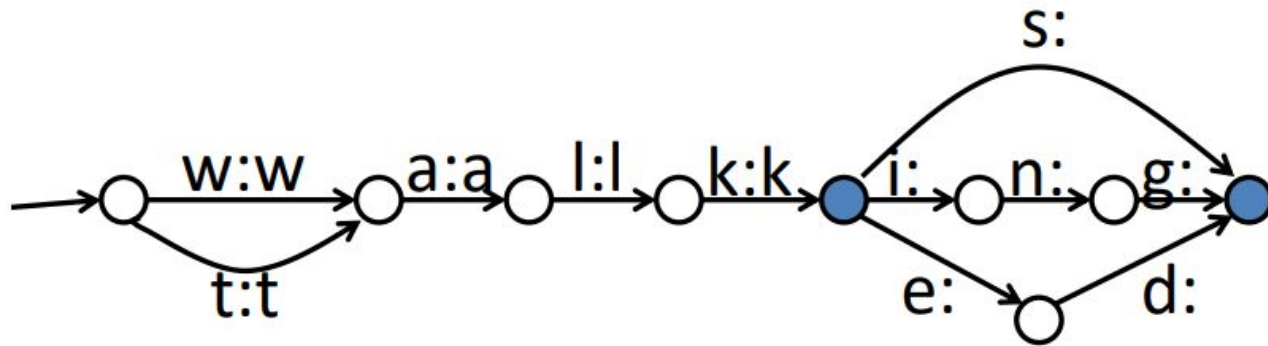


Both equivalent to the automaton on the previous slide

Finite State Transducers (FSTs)

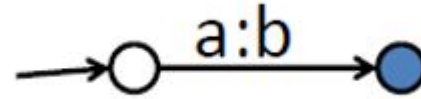
FSA → recognizes strings
FST → maps strings to strings

- FSTs are FSAs whose transitions are labelled with symbol pairs
- They map strings to (sets of) other strings



- Maps **walk, walks, walked, walking to walk**
- And **talk, talks, talked, talking to talk** (in generation mode)
- Can also map walk to walk, walks, walked, walking in analysis mode

FSTs and Regular Expressions



- Single symbol mapping $a:b$
- **Operations on FSTs:**
- Concatenation, Kleene's star, disjunction, conjunction, complement (from FSAs)
- Composition $A \parallel B$ (The output of transducer A is the input of transducer B.)
- **Projection**
 - Upper language replaces transition label $a:b$ by $b:b$
 - Lower language replaces transition label $a:b$ by $a:a$
 - The result corresponds to an automaton

Operations on FSAs

- Concatenation $A B$
- Optionality $A? = (| A)$
- Kleene's star $A^* = (| A | AA | AAA | \dots)$
- Disjunction $A | B$
- Conjunction $A \& B$
- Complement $!A$
- Subtraction $A - B = A \& !B$
- Reversal

Weighted Transducers

- A weighted FST **assigns a numerical weight** to each transition
- The total weight of a string-to-string mapping is the **sum of the weights on the corresponding path** from start to end state.
- Weighted FSTs allow **disambiguation** between different analyses by choosing the one with the smallest (or largest) weight.

Working with FSTs

- FSTs can be **specified by** means of regular expressions (like FSAs). The translation is performed by a compiler.
- Using the same algorithms as for FSA
 - FSTs can be made **epsilon-free** in the sense that no transition is labelled with $\epsilon:\epsilon$ (a pair of empty string symbols)
 - FSTs can be made **deterministic** in the sense that no two transitions originating in the same state have the same label pair
 - FSTs can be **minimised** in the sense that no other FST which produces the same regular relation with the same input-output alignment is smaller. (There might be a smaller transducer producing the same relation with a different alignment - No redundant equivalent states)
 - **FSTs can be used in both directions (generation and analysis)**

Working with FSTs: Examples

Consider, **Original (with $\epsilon:\epsilon$)**:

- Transition: $q_0 \xrightarrow{\epsilon:\epsilon} q_1$
- Transition: $q_1 \xrightarrow{a:\epsilon} q_1$
- Transition: $q_1 \xrightarrow{b:b} q_1$

Epsilon-free version

We **remove the $\epsilon:\epsilon$ transition** by merging states q_0 and q_1 .

After elimination:

- Transition: $q_0 \xrightarrow{a:\epsilon} q_0$
 - Transition: $q_0 \xrightarrow{b:b} q_0$
- ✓ Same input-output relation
✓ No transition labeled $\epsilon:\epsilon$

Working with FSTs: Examples

An FST that maps lowercase letters to uppercase:

- **Non-deterministic FST:**

From state q_0 :

$q_0 \xrightarrow{a:A} q_1$

$q_0 \xrightarrow{a:A} q_2$

- Two transitions from the same state have the **same label pair (a:A)**, so it's non-deterministic.
- **Deterministic version**
 - Merge q_1 and q_2 :
 - From state q_0 :
 - $q_0 \xrightarrow{a:A} q_1$
 - Exactly one transition per label pair, & Deterministic FST

Working with FSTs: Examples

An FST that maps:

$ab \rightarrow xy$

Non-minimal FST:

$q_0 \xrightarrow{a:x} q_1 \xrightarrow{b:y} q_2$

$q_0 \xrightarrow{a:x} q_3 \xrightarrow{b:y} q_4$

- States q_1 and q_3 are equivalent
- States q_2 and q_4 are equivalent

Minimal FST

Merge equivalent states:

$q_0 \xrightarrow{a:x} q_1 \xrightarrow{b:y} q_2$

- ✓ Same regular relation
- ✓ Same input-output alignment
- ✓ Fewer states $\rightarrow$ **minimal**

Parsing

- Taking a surface **input and identifying its components** and underlying structure
- **Morphological parsing:** **parsing a word into stem and affixes** and identifying the parts and their relationships
- **Stem and features:**
 - **goose -> goose +N +SG or goose +V**
 - **geese -> goose +N +PL**
 - **gooses -> goose +V +3SG**
- Bracketing: indecipherable -> [in [[de [cipher]] able]]

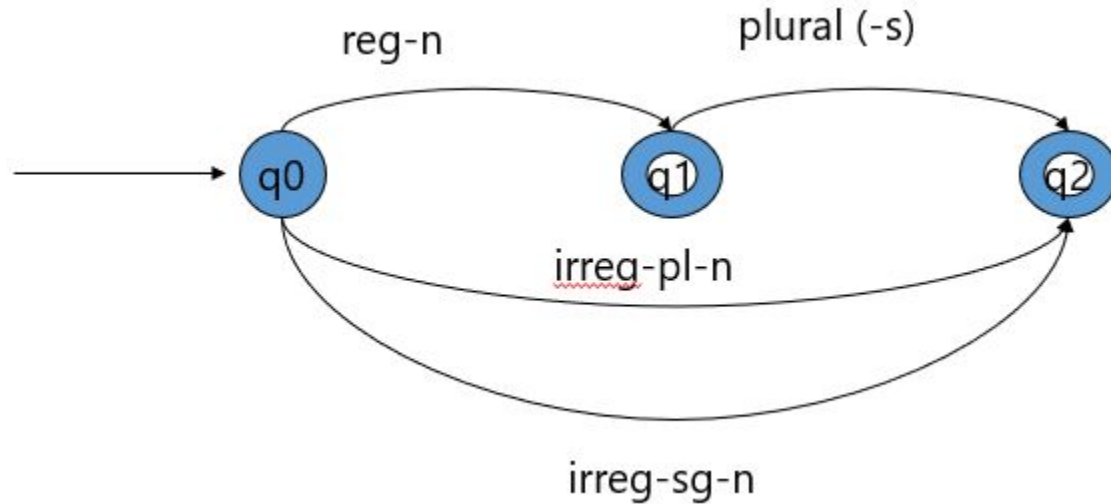
Finite State Morphological Parsing

- **Morphological Parsing:**
 - **takes** : root=take, category=V, person=3, number=sg
 - **cities** : root=city, category=N, number=plural
 - **ships**: a. root=ship, category=N, number=plural
 - b. root=ship, category=V, person=3, number=sg
- What does a parser need?
 - **Lexicon**: root forms of the words in a language
 - **Morphological Rules**: morpheme ordering rules
 - **Orthographic Rules**: spelling change rules
- Let us represent each of these components as finite-state automata.

Morphotactic Models

- English nominal inflection

- Inputs: cats, goose, geese





Antworth data on English Adjectives

- Big, bigger, biggest
- Cool, cooler, coolest, coolly
- Red, redder, reddest
- Clear, clearer, clearest, clearly, unclear, unclearly
- Happy, happier, happiest, happily
- Unhappy, unhappier, unhappiest, unhappily
- Real, unreal, really

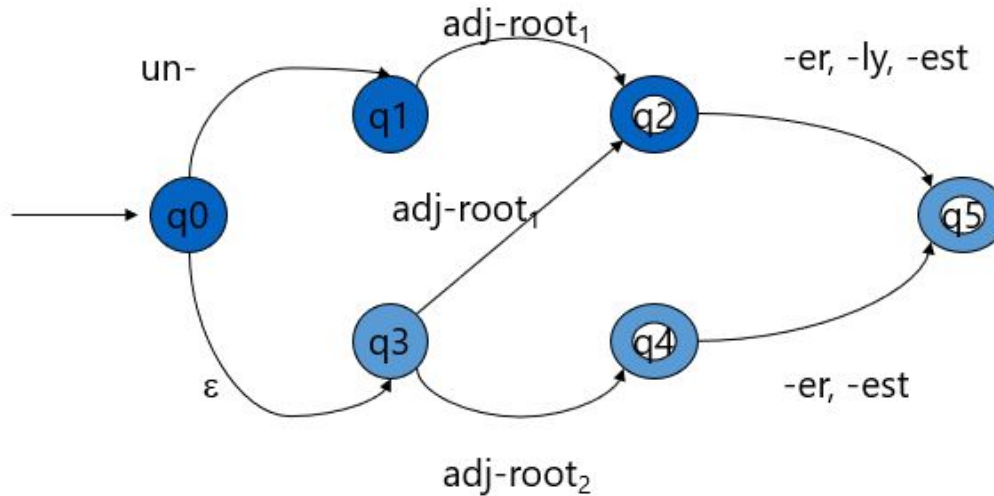


Antworth data on English Adjectives

- **Big**, **bigger**, **biggest**
- **Cool**, **cooler**, **coolest**, **coolly**
- **Red**, **redder**, **reddest**
- **Clear**, **clearer**, **clearest**, **clearly**, **unclear**, **unclearly**
- **Happy**, **happier**, **happiest**, **happily**
- **Unhappy**, **unhappier**, **unhappiest**, **unhappily**
- **Real**, **unreal**, **really**

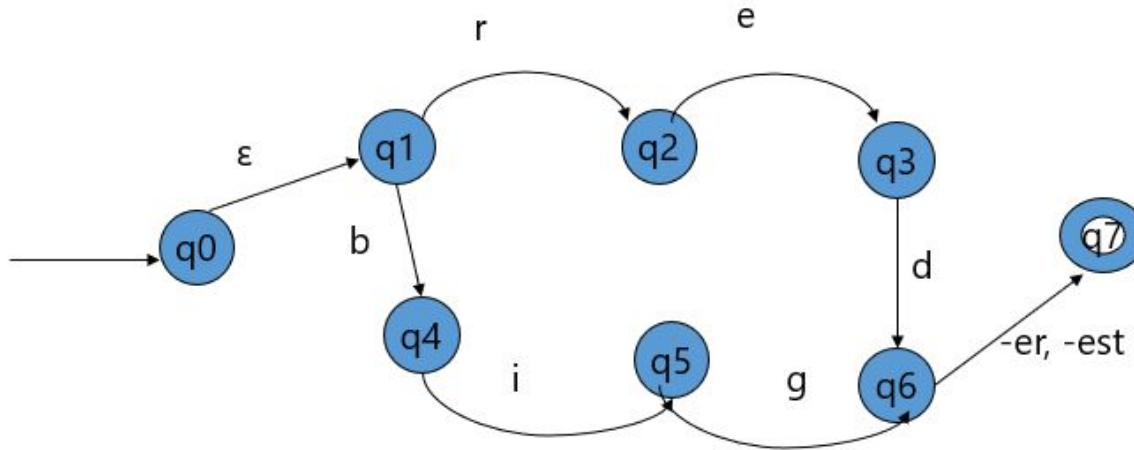
Derivational Morphology: Adjective Fragment

- Adj-root₁: clear, happy, real
- Adj-root₂: big, red



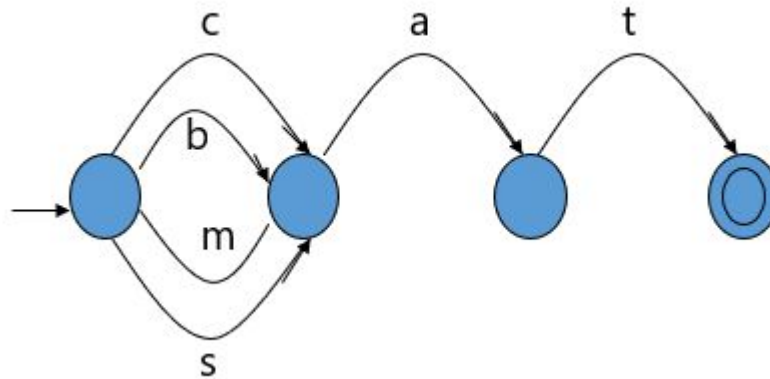
Using FSAs to Represent the Lexicon and Do Morphological Recognition

- **Lexicon:** We can expand each non-terminal in our NFSA into each stem in its class (e.g. $\text{adj_root}_2 = \{\text{big}, \text{red}\}$) and expand each such stem to the letters it includes (e.g. $\text{red} \rightarrow \text{r e d}$, $\text{big} \rightarrow \text{b i g}$)



Lexicon as an FSA

- Words can be represented as paths in a FSA with characters as transition symbols.
 - Also **called a “trie” structure**
 - **List of words:** bat, cat, mat, sat



Parsing / Generation Vs Recognition

- Recognition is usually not quite what we need.
- Usually if we find some string in the language we need to find the structure in it (parsing)
- Or we have some structure and we want to produce a surface form(production/generation)
- **Example :** From “cats” to “cat +N +PL” - Parsing

Parsing with Finite State Transducers

- **cats** -> **cat** +**N** +**PL**
- **Kimmo Koskenniemi two-level morphology:**
 - Words represented as correspondences between **lexical** level (the morphemes) and **surface** level (the orthographic word)
 - **Morphological parsing:** building **mappings** between the lexical and surface levels

Lexical Level

Surface Level

	c	a	t	+N	+PL	
	c	a	t	s		

Finite State Transducers

- **FSTs map between one set of symbols and another** using an FSA whose alphabet S is composed of pairs of symbols from input and output alphabets
- In general, **FSTs can be used for**
 - Translator (Hello : Ciao)
 - Parser/generator (Hello : How may I help you?)
 - To map between the lexical and surface levels of Kimmo's 2-level morphology

Finite State Transducers

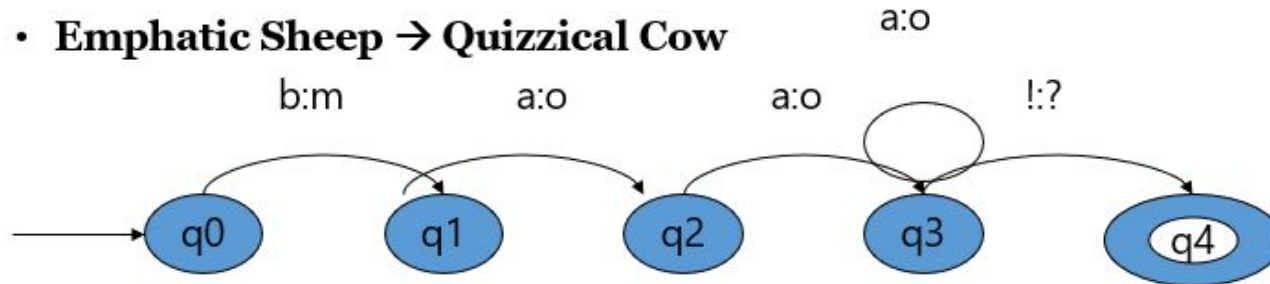
- Finite State **Acceptors** represent regular sets.
- Finite State **Transducers** represent regular relations.
- **Relation:**
 - If A and B are two regular sets; relation $R \subseteq A \times B$
 - **Example:** $\{(x,y) \mid x \in a^*, y \in b^*\}$

Finite State Transducers

- The **simple story**
 - Add another tape
 - Add extra symbols to the transitions
 - On **one tape we read “cats”**, on the **other we write “cat +N +PL”**

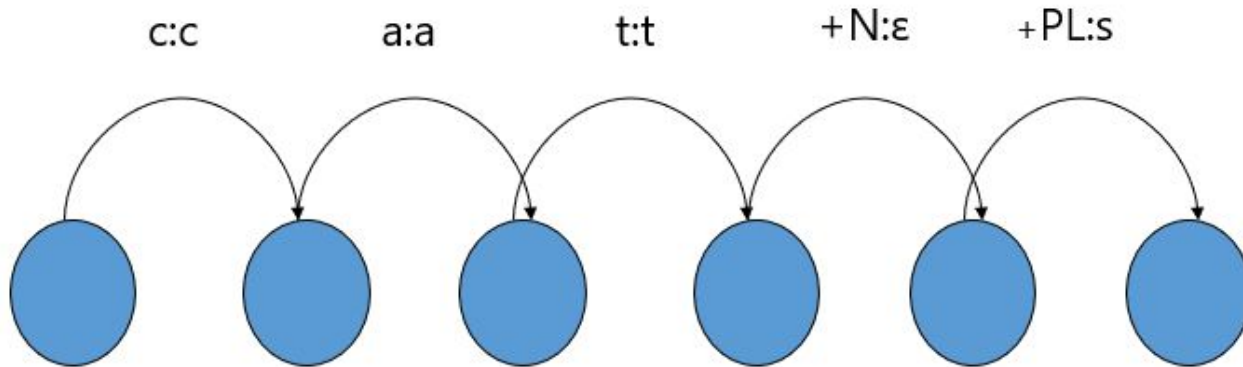
FST is a 5-tuple consisting of

- Q : set of states $\{q_0, q_1, q_2, q_3, q_4\}$
- Σ : an alphabet of complex symbols, each an i/o pair s.t. $i \in I$ (an input alphabet) and $o \in O$ (an output alphabet) and Σ is in $I \times O$
- q_0 : a start state
- F : a set of final states in Q $\{q_4\}$
- $\delta(q, i: o)$: a transition function mapping $Q \times \Sigma$ to Q
- **Emphatic Sheep $\rightarrow$ Quizzical Cow**



Transitions

- $c:c$ means read a c on one tape and write a c on the other
- $+N:\epsilon$ means read a $+N$ symbol on one tape and write nothing on the other
- $+PL:s$ means read $+PL$ and write an s



Orthographic Rules and FSTs

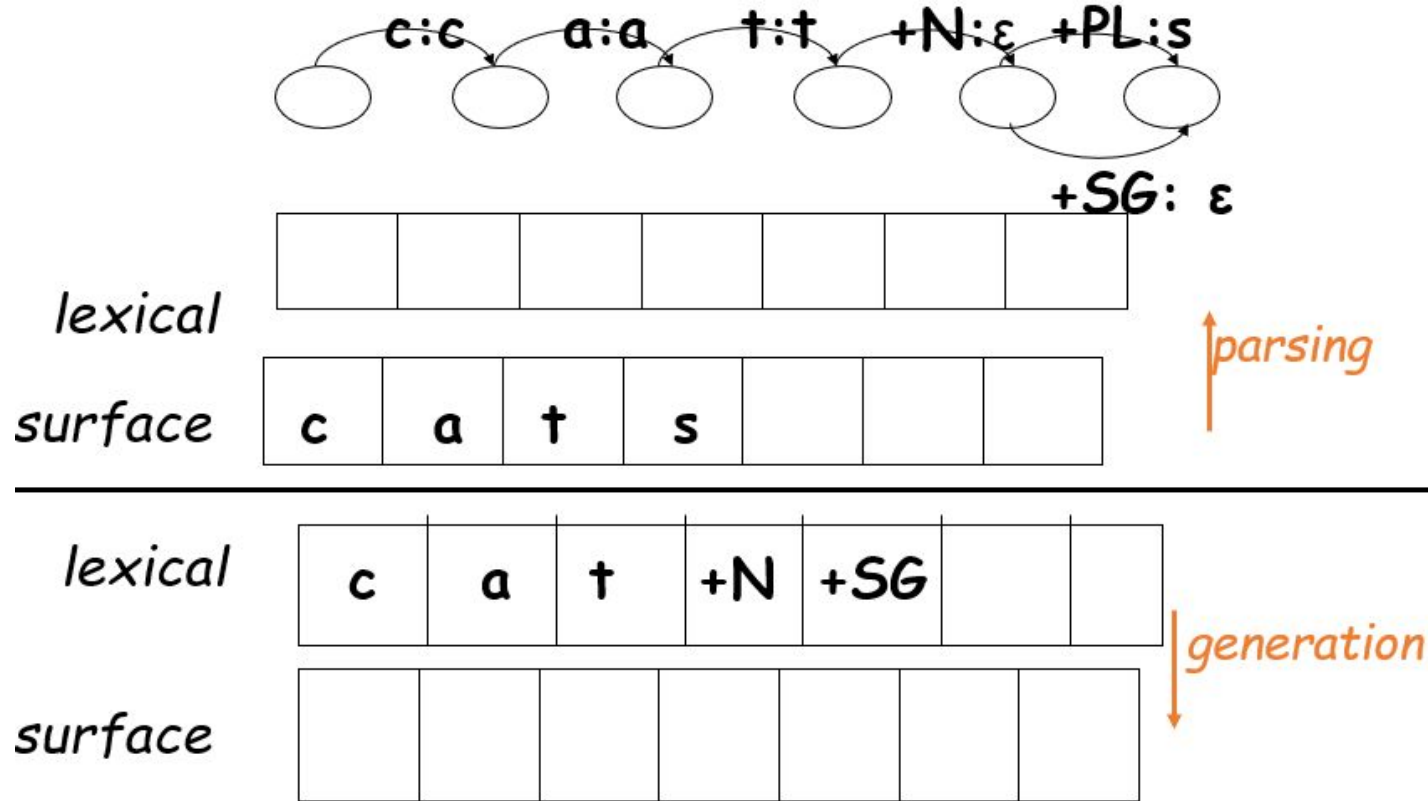
- Define additional FSTs to implement rules such as consonant doubling (**beg** -> **begging**), 'e' deletion (**make** -> **making**), 'e' insertion (**watch** -> **watches**), etc.

Lexical	f	o	x	+N	+PL	
Intermedi ate	f	o	x	^	s	#
Surface	f	o	x	e	s	

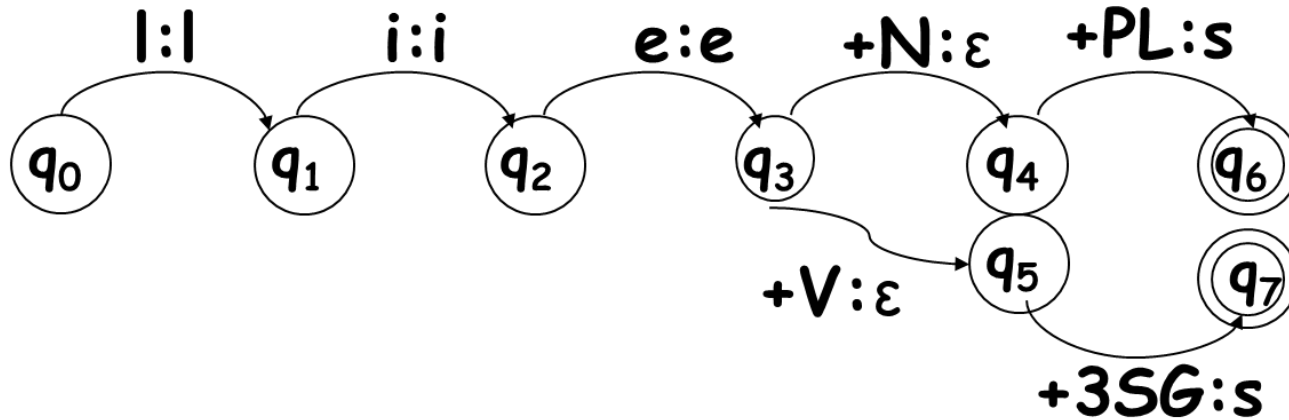
FSTs can be used as...

- **Translators**: input one string from I , output another from O (or vice versa)
- **Recognizers**: input a string from $I \times O$
- **Generator**: output a string from $I \times O$

Examples as a Translator..



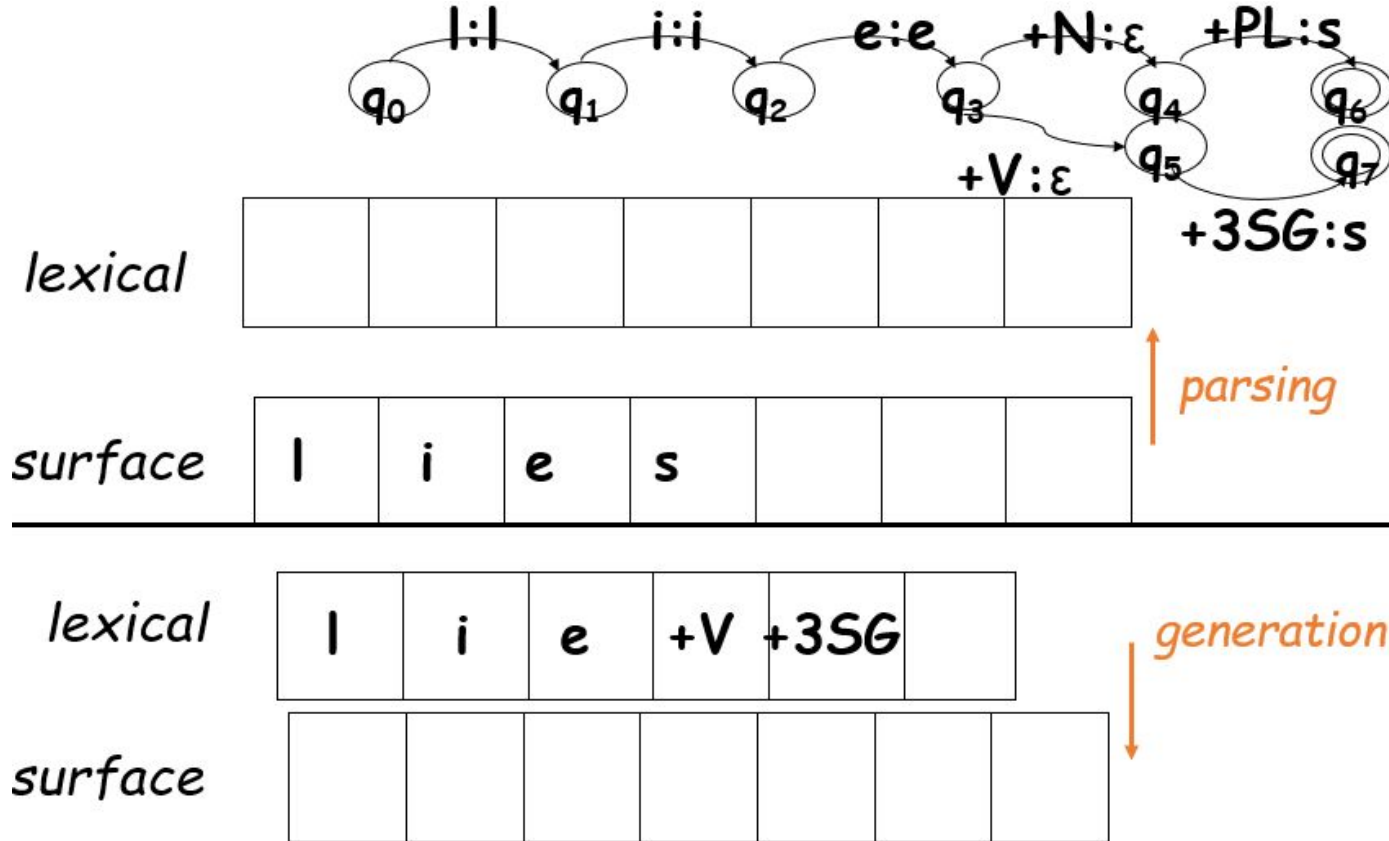
Slightly More complex Example



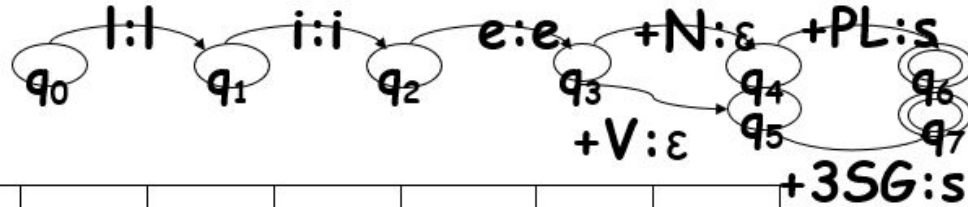
Transitions (as a translator):

- **l:l** means read an **l** on one tape and write an **l** on the other (or vice versa)
- **+N:ε** means read a **+N** symbol on one tape and write nothing on the other (or vice versa)
- **+PL:s** means read **+PL** and write an **s** (or vice versa)

Examples (as a Translator..)



Examples (as a recognizer and a generator)



lexical

l	i	e	+V	+3SG		
---	---	---	----	------	--	--

surface

l	i	e	s			
---	---	---	---	--	--	--

lexical

--	--	--	--	--	--	--

surface

--	--	--	--	--	--	--

More FST Examples...

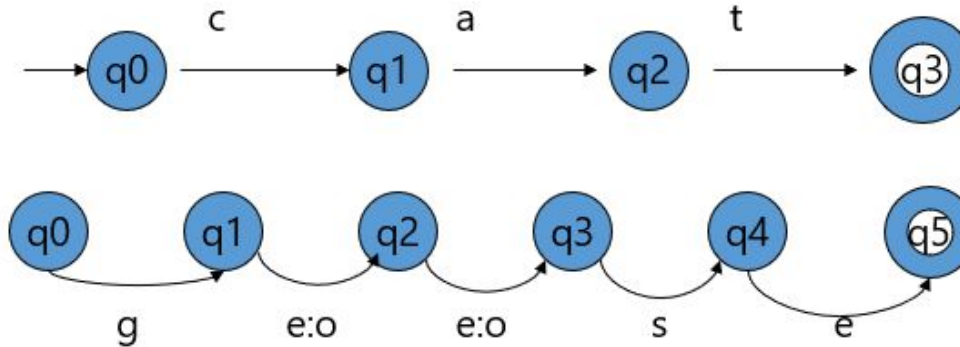
- **Speech Recognition** as FST composition
 - A: Acoustic $\rightarrow$ phones
 - Lex: phones $\rightarrow$ Words
 - Gram: Words $\rightarrow$ Words
- Speech Recognition: $A * Lex * Gram$
- Weighted FSTs **allow for ranking of the generated outputs.**

Limitations

- To cover all of e.g. English will **require very large FSAs** with consequent search problems
 - Adding new items to the lexicon means **re-computing the FSA**
 - **Non-determinism**
- FSAs can only tell us whether a word is in the language or not – what if we want to know more?
 - What is the stem?
 - What are the affixes and what sort are they?
 - We used this information to build our FSA: can we get it back?

FST for a 2-Level Lexicon

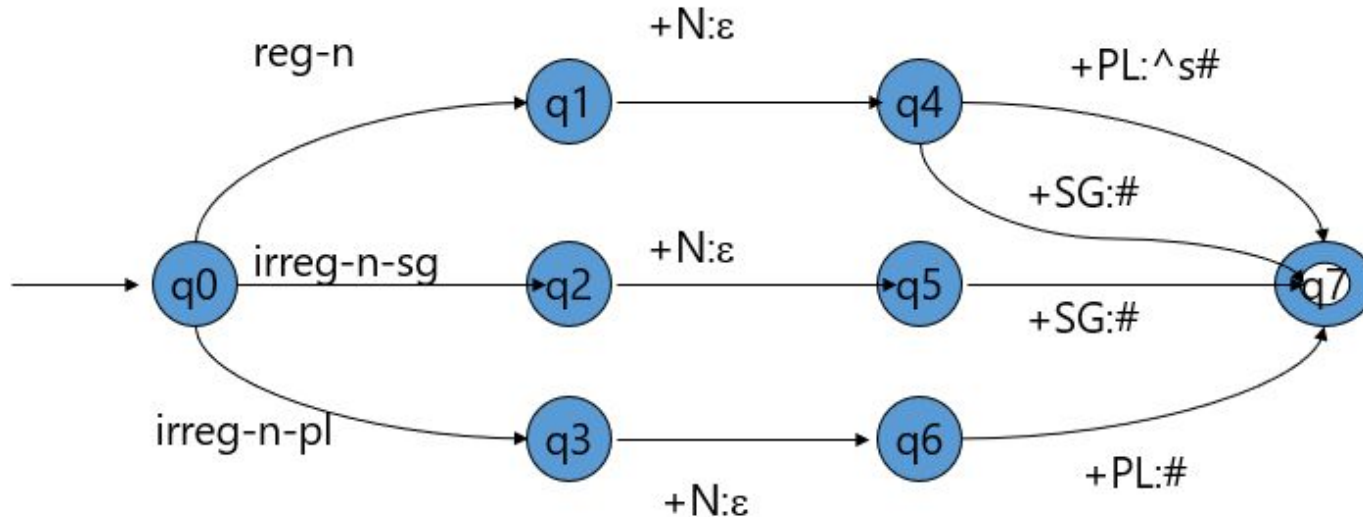
E.g.



<u>Reg-n</u>	<u>Irreg-pl-n</u>	<u>Irreg-sg-n</u>
cat	go:e o:e se	go <u>o</u> se

FST for English Nominal Inflection

- Combining (cascade or composition) this FSA with FSAs for each noun type replaces **e.g. reg-n with every regular noun** representation in the lexicon





Multi-Tape Machines

- To deal with this we can simply **add more tapes and use the output of one tape machine as the input to the next**
- So to handle irregular spelling changes we'll add **intermediate tapes with intermediate symbols**

Multi-Tape Machines

- We use **one machine to transduce between the lexical and the intermediate level**, and another to handle the **spelling changes to the surface tape**

Lexical

	f	o	x	+N	+PL			
--	----------	----------	----------	-----------	------------	--	--	--

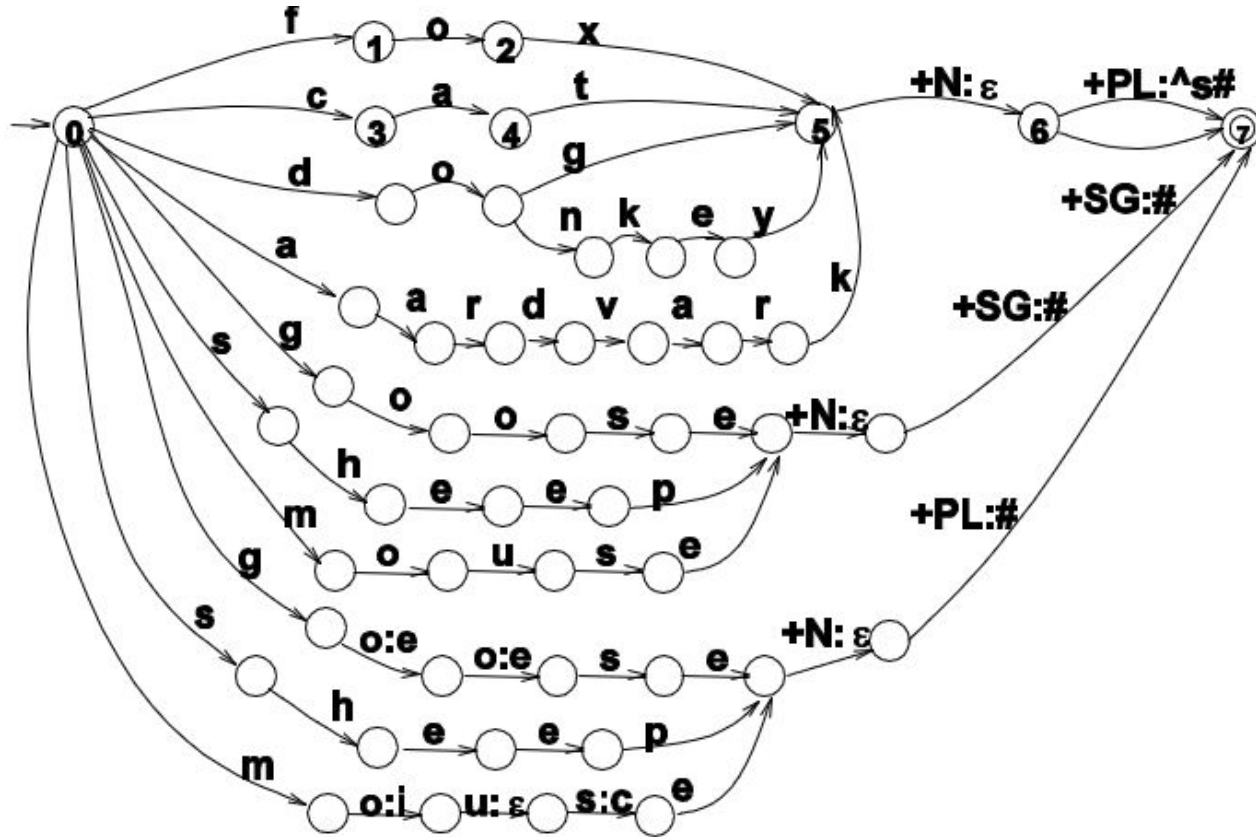
Intermediate

	f	o	x	^	s	#		
--	----------	----------	----------	----------	----------	----------	--	--

Surface

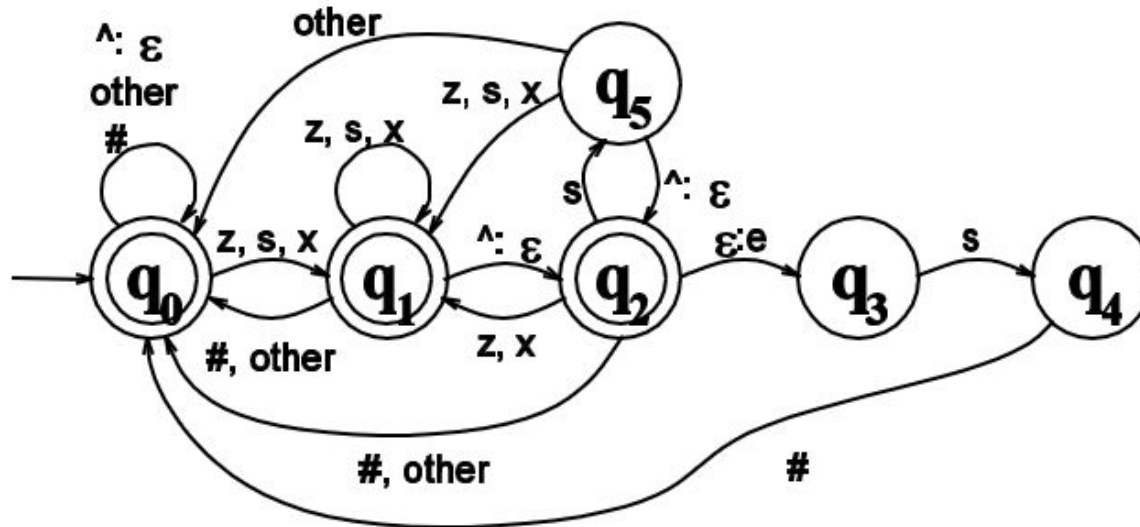
	f	o	x	e	s			
--	----------	----------	----------	----------	----------	--	--	--

Lexical to Intermediate Level



Intermediate to Surface Level

- The add an “e” rule as in $\text{fox}^s\# \leftrightarrow \text{foxes}$



Parsing with FST lexicon and rules

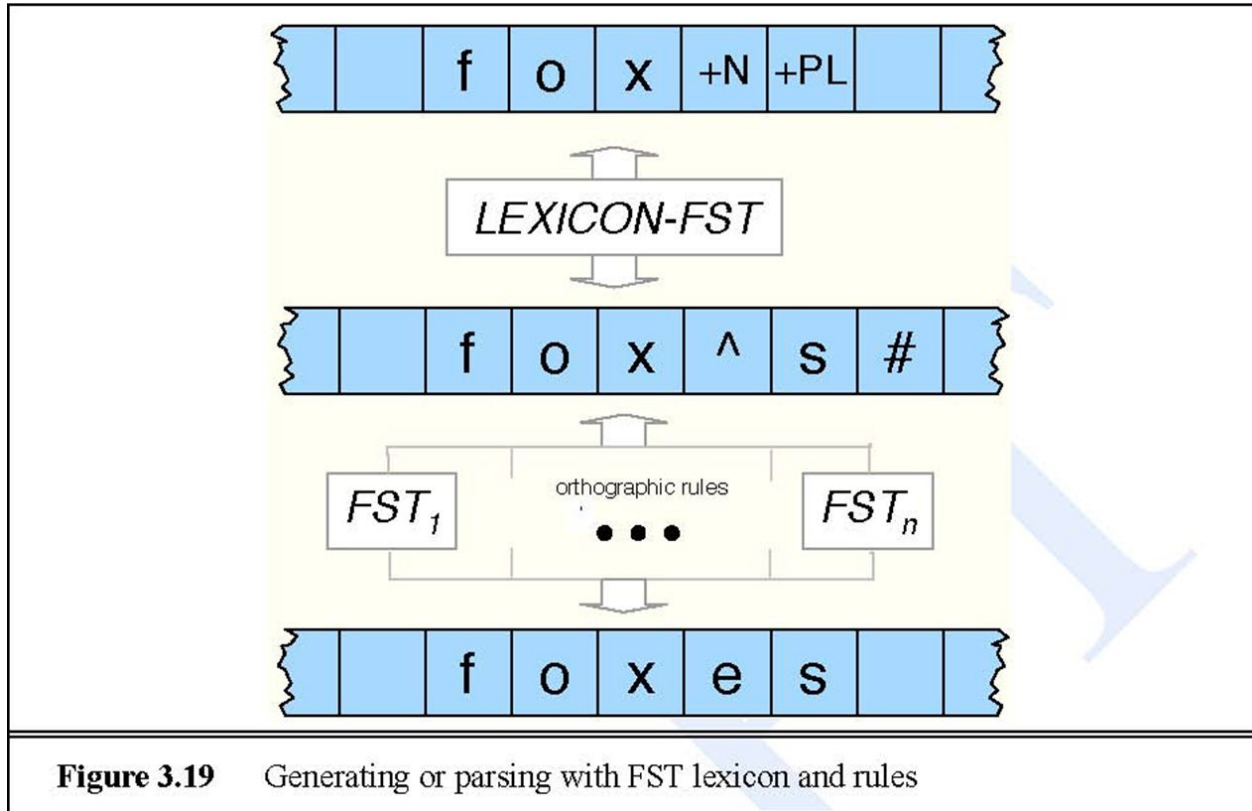


Figure 3.19 Generating or parsing with FST lexicon and rules

Parsing with FST lexicon and rules

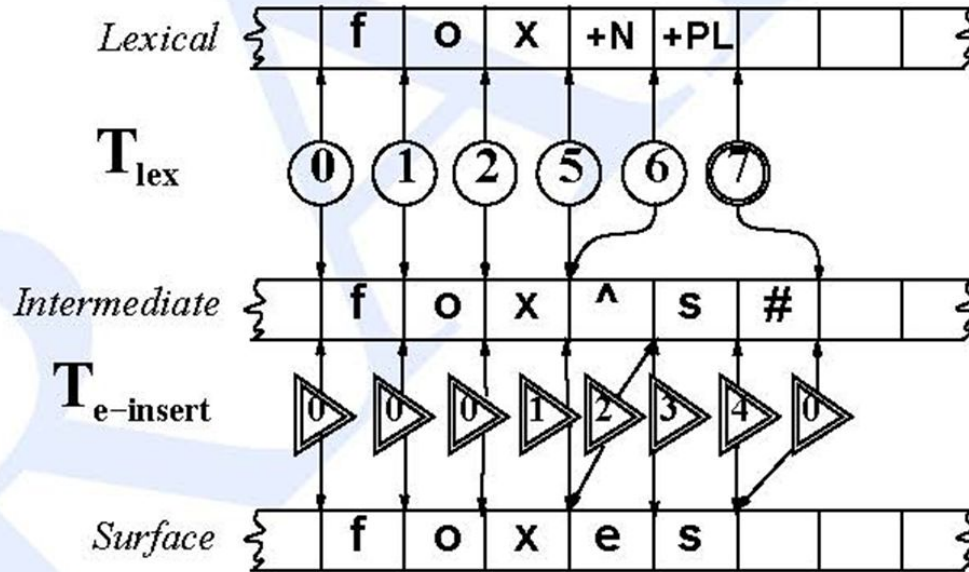


Figure 3.20 Accepting *foxes*: The lexicon transducer T_{lex} from Fig. 3.14 cascaded with the E-insertion transducer in Fig. 3.17.